

Proyecto de Simulación

Sistema de Tiempo Real: Controlador Simplificado de Bomba de Infusión

Levis Joaquin

Picco Valentino



Índice

Índice.....	1
Descripción del problema.....	2
Parámetros del sistema.....	2
Especificación DEVS de los modelos con notación CML-DEVS.....	3
Generador de órdenes médicas.....	3
Sensor de flujo.....	5
Controlador de bomba.....	6
Modelo Acoplado A1.....	10
Actuador de la bomba.....	11
Módulo de alarmas.....	13
Logger.....	15
Modelo Acoplado A2.....	17
Aclaración.....	17
Implementación en código.....	18
Primeros modelos atómicos (Generador, Sensor y Controlador).....	18
Primeros test unitarios.....	18
Funcionamiento normal, sin fallas.....	18
Cambio de orden médica durante la infusión.....	19
Orden médica con caudal igual a cero.....	19
Desvío de caudal mayor al 10% durante más de 5 segundos.....	19
Fin de bolsa con confirmación del enfermero.....	19
Implementación del resto de modelos atómicos (Actuador, Logger y Alarmas).....	20
Nuevos test unitarios y cobertura los demás escenarios.....	20
Desvío leve de caudal que es corregido por el controlador.....	20
Alarma crítica no confirmada durante 30 segundos.....	21
¿Qué nos falta?.....	21



Descripción del problema

Se desea modelar y simular un sistema automático de administración de medicación intravenosa. El sistema controla una bomba de infusión que debe suministrar una droga a un paciente respetando una dosis objetivo indicada por una orden médica. El sistema debe recibir órdenes médicas, controlar el caudal administrado, detectar desviaciones persistentes respecto del caudal indicado y reaccionar ante la condición de fin de bolsa. También deberá gestionar alarmas asociadas a dichas situaciones. El objetivo principal es estudiar el comportamiento temporal del sistema y verificar que las acciones de control ocurran dentro de los tiempos especificados.

Parámetros del sistema

Los parámetros del sistema especificados en el enunciado original, junto a sus valores sugeridos en el mismo, son los siguientes:

Parámetro	Valor sugerido
Rango de caudal permitido	0 a 200 ml/h
Tiempo máximo para iniciar infusión	3 s
Período de muestreo del sensor de flujo	1 s
Latencia del actuador de la bomba	0.5 s
Desvío máximo permitido de caudal	10%
Tiempo máximo de desvío de caudal	5 s
Tiempo para confirmar alarma crítica	30 s
Período de repetición de alarma crítica	10 s
Tiempo máximo luego de fin de bolsa	60 s



Especificación DEVS de los modelos con notación

CML-DEVS

Generador de órdenes médicas

Genera eventos *ordenMedica* que indican el caudal objetivo que debe administrar la bomba.

Las órdenes se generarán de manera determinística.

atomic *generadorDeOrdenesMedicas* is $\langle X, Y, S, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$ where

X is

$\emptyset$; // El generador no recibe eventos externos

end X

Y is

out_orden : $[0,200]$;

end Y

S is

s : $[0,200] \times R$;

end S



$\delta_{\text{int}}((\text{caudal}, \sigma))$ is

defcases

$s = (\text{caudal}', \sigma')$ //El sistema pasa a un nuevo estado con un caudal y tiempo

calculado

defcases

end δ_{int}

δ_{ext} is

$\emptyset$ // El generador no recibe eventos externos por lo cual no se define una función de

transición externa

end δ_{ext}

$\lambda((\text{caudal}, \sigma))$ is

defcases

case out = caudal;

end defcases

end λ

$\text{ta}((\text{caudal}, \sigma))$ is

σ ;

end ta

end atomic



Sensor de flujo

Mide periódicamente el caudal real administrado por la bomba.

atomic *sensorDeFlujo* is $\langle X, Y, S, \delta_{\text{int}}, \delta_{\text{ext}}, \lambda, \tau_a \rangle$ where

X is

in_caudal : $[0, 200]$;

end X

Y is

out_flujo : $[0, 200]$;

end Y

S is

s : $[0, 200] \times R$;

end S

$\delta_{\text{ext}}((\text{caudalMedido}, \sigma), e, x)$ is

s = (x, $\sigma - e$)

end δ_{ext}



$\lambda((\text{caudalMedido}, \sigma))$ is

defcases

case out = caudalMedido;

end defcases

end λ

$\delta\text{int}((\text{caudalMedido}, \sigma))$ is

defcases

s = (caudalMedido, 1);

end defcases

end δint

$\text{ta}((\text{caudalMedido}, \sigma))$ is

σ ;

end ta

Controlador de bomba

Es el componente central del sistema. Recibe las órdenes médicas, las mediciones del sensor de flujo, la señal de fin de bolsa y las confirmaciones del enfermero. Decide si debe iniciar la infusión, ajustar el caudal, detener la bomba o emitir alarmas.



atomic *controladorDeBomba* is $\langle X, Y, S, \delta_{\text{int}}, \delta_{\text{ext}}, \lambda, \tau_a \rangle$ where

X is

in_ordenMedica : [0,200];

in_sensorFlujo : [0,200];

in_finBolsa : {emitir};

in_confEnf : {emitir};

end X

Y is

out_ajustarCaudal : [0,200];

out_detenerBomba : {emitir};

out_alarmaBaja : {emitir};

out_alarmaMedia : {emitir};

out_alarmaCritica : {emitir}; //El criterio para pasar de alarma media a crítica fue de

5 segundos después de que se produzca la alarma media

end Y

S is

s : {ajustando, suspendido, fin_bolsa, reemplazando bolsa, bloqueando, bloqueado} ×

$[0,200] \times R \times \text{Salida} \times (R \cup \{\infty\})$

end S



$\delta_{\text{ext}}((\text{fase}, \text{caudalObj}, \text{tiempDesvio}, \text{salida}, \sigma), e, (\text{puerto}, x))$ is

defcases

case $s = (\text{ajustando}, x, \text{tiempDesvio}, (\text{out_ajustarCaudal}, x), 3)$; if $(\text{puerto} = \text{in_ordenMedica} \text{ and } x > 0 \text{ and } \text{fase} \neq \text{reemplazando_bolsa})$

case $s = (\text{suspendido}, 0, 0, (\text{out_detenerBomba}, \text{emitir}), 0)$; if $(\text{puerto} = \text{in_ordenMedica} \text{ and } x = 0)$

case $s = (\text{fase}, \text{caudalObj}, 0, \emptyset, \sigma - e)$; if $(\text{puerto} = \text{in_sensorFlujo} \text{ and } |x - \text{caudalObj}| \leq 0.1 * \text{caudalObj})$

case $s = (\text{fase}, \text{caudalObj}, \text{tiempdesvio} + 1, \emptyset, \sigma - e)$; if $(\text{puerto} = \text{in_sensorFlujo} \text{ and } |x - \text{caudalObj}| > 0.1 * \text{caudalObj} \text{ and } \text{tiempdesvio} < 5)$

case $s = (\text{fase}, \text{caudalObj}, 6, (\text{out_alarmaMedia}, \text{emitir}), 0)$; if $(\text{puerto} = \text{in_sensorFlujo} \text{ and } |x - \text{caudalObj}| > 0.1 * \text{caudalObj} \text{ and } \text{tiempdesvio} = 5)$

case $s = (\text{fase}, \text{caudalObj}, \text{tiempdesvio} + 1, \emptyset, \sigma - e)$; if $(\text{puerto} = \text{in_sensorFlujo} \text{ and } |x - \text{caudalObj}| > 0.1 * \text{caudalObj} \text{ and } \text{tiempdesvio} \geq 6 \text{ and } \text{tiempdesvio} < 9)$

case $s = (\text{bloqueando}, \text{caudalObj}, 10, (\text{out_alarmaCritica}, \text{emitir}), 0)$; if $(\text{puerto} = \text{in_sensorFlujo} \text{ and } |x - \text{caudalObj}| > 0.1 * \text{caudalObj} \text{ and } \text{tiempDesvio} = 9)$

case $s = (\text{fin_bolsa}, \text{caudalObj}, \text{tiempdesvio}, (\text{out_alarmaBaja}, \text{emitir}), 0)$; if $(\text{puerto} = \text{in_finBolsa} \text{ and } \text{fase} = \text{ajustando})$

case $s = (\text{suspendido}, 0, 0, \emptyset, \infty)$; if $(\text{puerto} = \text{in_confEnf} \text{ and } \text{fase} = \text{bloqueado})$

case $s = (\text{fase}, \text{caudalObj}, \text{tiempdesvio}, \text{salida}, \sigma - e)$; caso contrario



```

        end defcases

    end  $\delta_{\text{ext}}$ 

 $\lambda((\text{fase}, \text{caudalObj}, \text{tiempDesvio}, \text{salida}, \sigma))$  is

    defcases

        case out = salida;

    end defcases

end  $\lambda$ 

 $\delta_{\text{int}}((\text{fase}, \text{caudalObj}, \text{tiempDesvio}, \text{salida}, \sigma))$  is

    defcases

        case s = (suspendido, 0, 0,  $\emptyset$ ,  $\infty$ ); if (fase = suspendido)

        case s = (ajustando, caudalObj, tiempDesvio,  $\emptyset$ ,  $\infty$ ); if (fase = ajustando)

        case s = (fin_bolsa, caudalObj, tiempDesvio, (out_detenerBomba, emitir), 60);

    if (fase = fin_bolsa and salida = (out_alarmaBaja, emitir))

        case s = (reemplazando_bolsa, 0, 0,  $\emptyset$ , 120); if (fase = fin_bolsa and salida =
        (out_detenerBomba, emitir)) ) // Tras emitir la detención (pasaron los 60s), entra en
        recambio físico y carga 120s

        case s = (suspendido, 0, 0,  $\emptyset$ ,  $\infty$ ); if (fase = reemplazando_bolsa)

        case s = (bloqueado, caudalObj, tiempDesvio, (out_detenerBomba, emitir), 0);

    if (fase = bloqueando_bomba)

        case s = (bloqueado, caudalObj, tiempDesvio,  $\emptyset$ ,  $\infty$ ); if (fase = bloqueado)

    end defcases

end  $\delta_{\text{int}}$ 

```



ta((fase, caudalObj, tiempDesvio, salida, σ)) is

σ ;

end ta

Modelo Acoplado A1

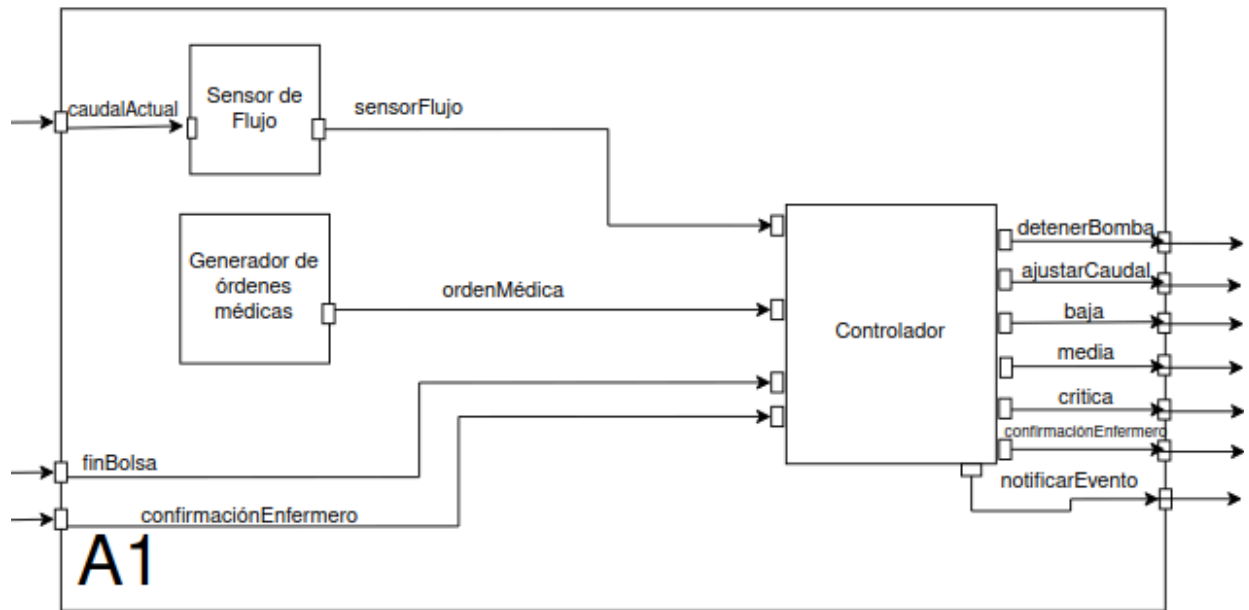


Figura 1: Modelo Acoplado A1

A raíz de las especificaciones realizadas hasta el momento de los modelos atómicos, es que podemos realizar el Modelo Acoplado A1 (Figura 1).



Actuador de la bomba

Representa el mecanismo físico que administra la medicación al paciente. Recibe órdenes del controlador y modifica el caudal real.

atomic *actuadorDeLaBomba* is $\langle X, Y, S, \delta_{int}, \delta_{ext}, \lambda, \tau_a \rangle$ where

X is

in_ajustarCaudal : [0,200];
in_detenerBomba : {emitir};

end X

Y is

out_caudalActual : [0,200];

end Y

S is

$s : \{\text{estable, transición}\} \times [0,200] \times [0,200] \times (R \cup \{\infty\});$

end S

$\delta_{ext}((\text{fase, caudalActual, caudalObjetivo, } \sigma), e, (\text{puerto, } x))$ is

defcases

case $s = (\text{transicion, caudalActual, } x, 0.5)$; if (puerto = in_ajustarCaudal and

fase = estable)



```

        case s = (transicion, caudalActual, 0, 0.5); if (puerto = in_detenerBomba)
    end defcases
end  $\delta_{ext}$ 

```

```

 $\lambda((fase, caudalActual, caudalObjetivo, \sigma))$  is
    defcases
        case out = caudalObjetivo;
    end defcases
end  $\lambda$ 

```

```

 $\delta_{int}((fase, caudalActual, caudalObjetivo, \sigma))$  is
    defcases
        s = (estable, caudalObjetivo, caudalObjetivo,  $\infty$ );
    end defcases
end  $\delta_{int}$ 

```

```

 $ta((fase, caudalActual, caudalObjetivo, \sigma))$  is
     $\sigma$ ;
end ta

```



Módulo de alarmas

Gestiona las alarmas visuales y sonoras. Debe repetir las alarmas críticas no confirmadas.

atomic *moduloDeAlarmas* is $\langle X, Y, S, \delta_{int}, \delta_{ext}, \lambda, \tau_a \rangle$ where

X is

in_alarmaBaja : {emitir};

in_alarmaMedia : {emitir};

in_alarmaCritica : {emitir};

in_confirmaciónEnfermero : {emitir};

end X

Y is

out_notificaciónAlarma : {emitir};

end Y

S is

$s : \{\text{ninguna, baja, media, critica}\} \times \{\text{pasivo, emitir, esperar30, esperar10}\} \times ($

$R \cup \{\infty\})$;

end S

$\delta_{ext}((\text{tipo, fase, } \sigma), e, (\text{puerto, } x))$ is

defcases



```

        case s = (tipo, emitir, 0); if (puerto = in_alarmaBaja or in_alarmaMedia or
in_alarmaCritica)

        case s = (ninguna, pasivo, ∞); if (puerto = in_confirmaciónEnfermero)

    end defcases

end δext

```

$\lambda(\text{tipo}, \text{fase}, \sigma)$ is

```

    defcases

        case out = tipo;

    end defcases

end λ

```

$\delta\text{int}(\text{tipo}, \text{fase}, \sigma)$ is

```

    defcases

        case s =(critica, esperar30, 30) if (fase = emitir and tipo = critica);

        case s =(ninguna, pasivo, ∞) if (fase = emitir and tipo != critica);

        case s =(critica, esperar10,10) if (fase = esperar30 or fase = esperar10);

    end defcases

end δint

```

$\text{ta}(\text{tipo}, \text{fase}, \sigma)$ is

σ ;



end ta

Logger

Registrador de eventos

atomic *logger* is $\langle X, Y, S, \delta_{\text{int}}, \delta_{\text{ext}}, \lambda, ta \rangle$ where

X is

in_evento : {emitir};

end X

Y is

$\emptyset$; // No tiene salidas, es el destino final de la información

end Y

S is

$s : (\text{ListaDeEventos}) \times \{\infty\}$; // Guarda el historial y un tiempo infinito

end S

$\delta_{\text{ext}}((\text{historial}, \sigma), e, (\text{puerto}, x))$ is

defcases

case $s = (\text{historial} + x, \infty)$; if (puerto = in_evento)

end defcases



end δext

$\lambda((\text{historial}, \sigma))$ is

$\emptyset$; // Al no tener tiempo finito, nunca ejecuta salida

end λ

$\delta\text{int}((\text{historial}, \sigma))$ is

$\emptyset$; // No hay transiciones internas

end δint

$\text{ta}((\text{historial}, \sigma))$ is

∞ ; // Siempre está pasivo esperando que el controlador le envíe algo

end ta

end atomic



Modelo Acoplado A2

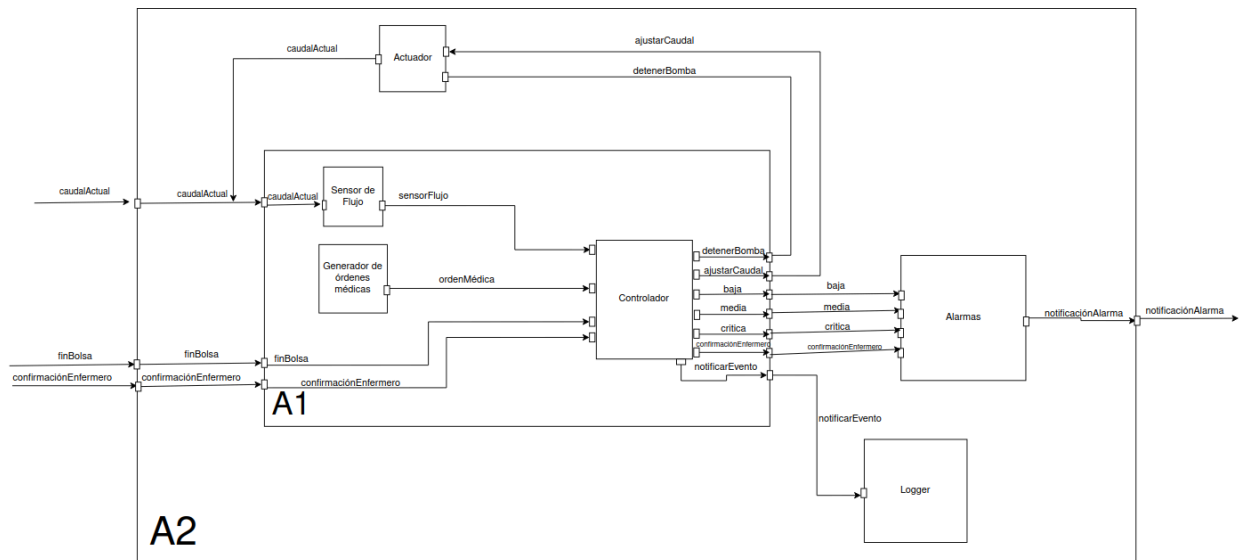


Figura 2: Modelo Acoplado A2

En conjunto con el Modelo A1 (Figura 1) y las especificaciones realizadas posteriormente del mismo, podemos acoplarlos para obtener el Modelo Acoplado A2 (Figura 2).

Aclaración

En caso de que alguna de las imágenes presentadas en este informe no se aprecien correctamente, puede hacer click [aquí](#) y será dirigido a un drive en donde podrá verlas de mejor manera.



Implementación en código

Para llevar a cabo la implementación decidimos utilizar *python* con el framework *PythonPDEVS* debido a su clara similitud entre el código y el formalismo DEVS, además de la facilidad para crear test unitarios para cada módulo con la librería *pytest*.

Primeros modelos atómicos (Generador, Sensor y Controlador)

Comenzamos igual que en la especificación, o sea con los modelos atómicos de *Generación de órdenes médicas*, *Sensor de flujo* y *Controlador de la bomba*, para luego continuar con el modelo acoplado A1 que une a los tres antes mencionados.

Primeros test unitarios

Una vez que tuvimos los primeros modelos atómicos implementamos algunos test con casos basados en los escenarios dentro del PDF del enunciado que podían realizarse hasta el momento, debido que aún nos faltaban el resto de modelos no podíamos implementar todos los escenarios pero nos sirvió para ver si la especificación y el código iban por buen camino. Los escenarios testeados en su momento fueron:

Funcionamiento normal, sin fallas

Archivo: *tests/test_controlador_de_bomba.py*

test_inicio_infusion: Verifica el pase a ajustando al recibir *ordenMedica*.

test_salida_ajustar_caudal: Verifica la orden hacia el "actuador" (que aún no existía).

test_post_ajuste_pasiva: Verifica que el controlador se quede esperando.



test_caudal_normal_no_alarma: Verifica que lecturas del Sensor normales no disparen nada.

Cambio de orden médica durante la infusión

Archivo: *tests/test_controlador_de_bomba.py*

test_cambio_orden_50_a_80: Verifica que enviar una nueva orden en pleno funcionamiento actualiza el caudalObj.

Orden médica con caudal igual a cero

Archivo: *tests/test_controlador_de_bomba.py*

test_desvio_dentro_margen_resetea : Verifica que el contador temporal de error vuelve a 0 si se normaliza.

test_desvio_exacto_margen : Verifica el límite de borde del 10%.

Desvío de caudal mayor al 10% durante más de 5 segundos

Archivo: *tests/test_controlador_de_bomba.py*

test_incremento_tiempDesvio

test_alarma_media_en_tiempDesvio_5

test_incremento_post_alarma_media

test_alarma_critica_en_tiempDesvio_9

test_bloqueando_a_bloqueado

test_confirmacion_enfermero_desbloquea

Fin de bolsa con confirmación del enfermero

Archivo: *tests/test_controlador_de_bomba.py*



test_fin_bolsa_en_ajustando
test_fin_bolsa_programa_detencion
test_fin_bolsa_a_reemplazando
test_orden_ignorada_en_reemplazando
test_reemplazando_a_suspendido

Implementación del resto de modelos atómicos (Actuador, Logger y Alarmas)

Ya teníamos una idea de como implementar y testear los modelos y escenarios, así que comenzamos con la implementación de los demás modelos atómicos: *Actuador de Bomba*, *Logger* y *Módulo de Alarmas* para luego conectarlos con A1 en la implementación del modelo acoplado A2.

Nuevos test unitarios y cobertura los demás escenarios

Ahora que contábamos con las demás implementaciones de los modelos atómicos podíamos testearlos para verificar los escenarios que antes nos faltaron o estaban testeados parcialmente:

Desvío leve de caudal que es corregido por el controlador

Archivo: *tests/test_controlador_de_bomba.py*

test_desvio_dentro_margen_resetea: Verifica el restablecimiento de contadores cuando el desvío se normaliza.



test_desvio_exacto_margen: Prueba de límite exacto del $\pm 10\%$.

Archivo: *tests/test_actuador_de_bomba.py*

test_ajuste_caudal_latencia y test_emision_caudal_transicion: Comprueban que el actuador ejecute la acción compensatoria aplicando el retraso de latencia mecánica de 0.5s.

Alarma crítica no confirmada durante 30 segundos

Archivo: *tests/test_modulo_de_alarmas.py*

test_emision_inmediata : El módulo despierta la sala sin retraso algorítmico al recibir evento grave.

test_ciclo_alarma_critica : Dispara el bucle de espera inicial de 30s (esperar30).

test_repeticion_alarma_critica y test_bucle_infinito_alarma : Prueba el ciclo estricto iterativo de repetición cada 10s (esperar10).

test_silencio_por_confirmacion : Apaga la secuencia únicamente ante intervención humana confirmada.

¿Qué nos falta?

Esta sección se va a borrar es solo para no olvidarnos de que cosas aún no están o deberíamos hacer.

- tests de acoplados
- menú
- parametrizar mejor (va de la mano con el menú)
- revisar la redacción del informe

